

Lycée Notre Dame de Boulogne

Classes de Première 02

**Numérique et Sciences Informatiques
(Cours)**

September 29, 2026

Auteur

Jim Pioche

Abstract

Ce document est un cours de niveau Première dans la matière NSI.

Contents

1	L’histoire de l’informatique	3
1.1	Avant les ordinateurs programmables	3
1.2	Les premières machines électroniques : naissance de l’ordinateur	4
1.3	L’invention des langages de programmation	5
1.4	De nos jours...l’intelligence artificielle	8
2	Représentation d’un nombre en machine	9
2.1	Ecriture d’un entier dans une base quelconque $b \geq 2$	9
2.2	Ecriture d’un entier d’une base quelconque vers une autre base	9
2.3	Règles d’addition des nombres binaires	10
2.4	Représentation en machine d’un entier positif	11
2.5	Représentation en machine d’un entier relatif	12
2.6	Représentation en machine d’un réel	15

1 L'histoire de l'informatique

L'informatique (néologisme inventé en 1962, contraction des mots automatique et information). En tant que discipline scientifique et technique, son histoire s'étale du 19^{ème} au 20^{ème} siècle.

Elle est intimement reliée au développement de la recherche Mathématiques en logique et en algorithmique.

1.1 Avant les ordinateurs programmables

Avant toute machine électronique, des mathématiciens, des marchands, des ingénieurs se sont posé des questions qui sont encore celles de l'informatique. On peut les regrouper en cinq problèmes, chacun ayant reçu une réponse avant 1940.

Problème 1 : calculer vite et sans se tromper. Les calculs à la main (impôts, comptabilité, navigation, astronomie) sont longs et les erreurs fréquentes. Les premières réponses sont des *outils* : le boulier ou l'abaque (dès l'Antiquité) qui matérialise les nombres, les bâtons de Napier (1617), puis la règle à calcul, qui a servi aux ingénieurs jusque dans les années 1970. Vient ensuite l'idée de confier le calcul à un mécanisme :

- **Blaise Pascal** construit en 1642 la *Pascaline*, une machine à roues dentées qui additionne et soustrait en propageant automatiquement les retenues. Il l'a conçue à dix-neuf ans pour aider son père, qui calculait les impôts de Rouen.
- **Gottfried Wilhelm Leibniz** en construit une version qui multiplie et divise (1673).

Que représentent ces machines dans l'histoire ? Ce sont des *calculatrices* mécaniques, pas des ordinateurs : elles effectuent *une* opération à la fois, et c'est l'humain qui décide de l'enchaînement des opérations. Elles prouvent qu'un raisonnement, ici l'arithmétique, peut être confié à un mécanisme, mais elles ne sont pas *programmables*.

Problème 2 : décrire une méthode de calcul sans ambiguïté. Pour faire calculer quelqu'un ou quelque chose, il faut expliquer précisément quoi faire, étape par étape. Euclide (vers -300) décrit ainsi le calcul du PGCD, et le mathématicien perse **Al-Khwârizmî** (IX^e siècle) rédige à Bagdad un traité de méthodes de résolution systématiques : son nom a donné le mot *algorithme*. Retenons l'idée d'une suite finie d'instructions élémentaires qui conduit toujours au résultat.

Problème 3 : enregistrer les instructions et enchaîner les opérations automatiquement. Comment faire pour qu'une machine exécute seule une suite d'opérations, et qu'on puisse changer cette suite sans reconstruire la machine ?

- **Joseph-Marie Jacquard** (1801) apporte la première réponse : ses métiers à tisser sont commandés par une chaîne de cartons perforés. Un trou laisse passer l'aiguille, l'absence de trou l'arrête, et le motif du tissu est dessiné automatiquement. Changer de carton change de motif : le *programme* est séparé de la machine et stocké sur un support.

- **Charles Babbage** (1834) imagine la *machine analytique*, en s'inspirant de ces cartes. Elle comprend un organe de calcul (le « moulin »), une mémoire (le « magasin »), des cartes pour les instructions et les données, et même des branchements conditionnels : on y reconnaît l'architecture d'un ordinateur. Elle ne sera jamais achevée, faute de moyens techniques et financiers.
- **Ada Lovelace**, mathématicienne, comprend en 1843 que cette machine ne sert pas qu'à calculer : elle peut manipuler tout ce qu'on sait représenter par des symboles. Elle écrit pour elle un programme de calcul des nombres de Bernoulli, considéré comme le **premier programme informatique**.

Problème 4 : représenter le raisonnement et l'information avec des symboles simples. Leibniz étudie en 1703 l'écriture des nombres en base 2 : deux symboles, 0 et 1, suffisent. **George Boole** montre en 1854 que le raisonnement logique se traite comme un calcul sur deux valeurs, vrai et faux (*l'algèbre de Boole*). Ces deux idées, qui semblent alors des curiosités mathématiques, sont exactement celles sur lesquelles reposeront les circuits électroniques et les conditions (**if**, **and**, **or**, **not**) de nos programmes. On peut y ajouter **Claude Shannon** qui montre en 1937 que des interrupteurs peuvent réaliser les opérations de Boole.

Problème 5 : traiter de grandes masses de données. Pour le recensement américain de 1890, **Herman Hollerith** enregistre les données de chaque habitant sur une carte perforée, et lit les cartes à l'aide de machines électriques : le dépouillement passe de sept ans à deux ans et demi. Son entreprise deviendra IBM. Ces machines *électromécaniques* (des pièces mécaniques commandées par des contacts et des relais électriques) sont une étape intermédiaire : elles ne sont pas programmables librement, et leur vitesse est limitée par le mouvement des pièces (quelques opérations par seconde). Elles annoncent pourtant l'industrie informatique. Des machines électromécaniques programmables voient encore le jour dans les années 1930-40, comme le *Z3* de Konrad Zuse (1941) ou le *Harvard Mark I* (1944), avant d'être remplacées par l'électronique.

Question finale : qu'est-ce que calculer ? En 1936, **Alan Turing** répond sans construire aucune machine : il imagine la *machine de Turing* (un ruban, une tête de lecture, quelques règles) et démontre que tout ce qui se calcule par une méthode systématique peut être calculé par une telle machine. Il montre aussi qu'il existe des problèmes qu'aucun programme ne pourra jamais résoudre. C'est le fondement théorique de l'informatique.

1.2 Les premières machines électroniques : naissance de l'ordinateur

Pourquoi l'électronique ? Les relais électromécaniques sont lents. Les *tubes à vide*, qui laissent ou non passer un courant sans aucune pièce mobile, permettent de commuter des milliers de fois plus vite. La Seconde Guerre mondiale (calculs de trajectoires d'obus, décryptage des messages ennemis) fournit les moyens de les utiliser à grande échelle.

Les premières machines (1943 – 1948).

- *Colossus* (Royaume-Uni, 1943), construit pour décrypter les messages allemands, est le premier calculateur électronique programmable, mais il est spécialisé.

- *ENIAC* (États-Unis, 1945) est le premier grand ordinateur électronique généraliste : 30 tonnes, 18 000 tubes à vide, 5 000 additions par seconde. Pour changer de programme, il faut rebrancher des câbles à la main, ce qui prend des jours. Ce sont des femmes, les « ENIAC girls », qui le programment.
- *Manchester Baby* (1948) est le premier ordinateur à exécuter un programme stocké dans sa mémoire.

John von Neumann : l'architecture (1945). Le mathématicien hongro-américain John von Neumann décrit une organisation qui résout le problème du rebranchement de l'ENIAC : le *programme* est enregistré dans la mémoire, au même endroit que les données. La machine comprend une mémoire, une unité de calcul (UAL), une unité de commande qui lit et exécute les instructions une à une, et des dispositifs d'entrée-sortie. Changer de programme revient à écrire d'autres instructions en mémoire. Cette **architecture de Von Neumann** est toujours celle de nos ordinateurs et de nos téléphones.

Le mot « ordinateur ». En 1955, IBM France demande conseil à un professeur de lettres, Jacques Perret, pour trouver un nom français au mot *computer*. Il propose « ordinateur », un adjectif du vocabulaire religieux qui désigne « Dieu qui met de l'ordre dans le monde ». Les anglophones, eux, disent *computer* (« calculateur »), ce qui rappelle que ces machines sont d'abord nées pour calculer.

La miniaturisation (1947 – 1971). Les tubes à vide sont volumineux, chauffent et tombent souvent en panne. Les étapes suivantes sont :

- 1947 : le *transistor* (Bardeen, Brattain, Shockley, laboratoires Bell), petit, fiable et économe, remplace le tube à vide ;
- fin des années 1950 : le *circuit intégré* (Kilby, Noyce) réunit des milliers de transistors sur une plaque de silicium ;
- 1971 : le *microprocesseur* Intel 4004 place toute l'unité de calcul sur une seule puce.

En 1965, Gordon Moore avait constaté que le nombre de transistors par puce doublait à peu près tous les deux ans. Cette *loi de Moore* s'est vérifiée pendant plusieurs décennies : c'est pourquoi votre téléphone est plus puissant que les ordinateurs qui ont envoyé l'homme sur la Lune.

L'ordinateur personnel et le réseau. Devenues petites et peu chères, les machines entrent dans les foyers : Apple II (1977), IBM PC (1981), Macintosh (1984). En parallèle, on relie les ordinateurs entre eux : le protocole *TCP/IP* (1983) est le langage commun d'*Internet*, et en 1989, au CERN, Tim Berners-Lee invente le *World Wide Web* (HTML, HTTP, URL). Il ne faut pas confondre Internet, le réseau, avec le Web, l'un de ses services.

1.3 L'invention des langages de programmation

Le problème. Un processeur ne comprend que des suites de 0 et de 1, son *langage machine*. Programmer directement ainsi est long et source d'erreurs. Toute l'histoire des

langages est celle d'une montée en abstraction : se rapprocher de la manière de penser de l'humain, tout en laissant la machine faire la traduction.

Les premières étapes. Vers 1945, l'ingénieur allemand Konrad Zuse invente *Plankalkül*, le tout premier langage de programmation conçu, mais il ne sera pas utilisé à l'époque. Sur les machines, on utilise d'abord le langage machine, puis l'*assembleur*, qui donne un nom lisible à chaque instruction (ADD, MOV, ...), mais qui reste propre à un processeur donné.

Grace Hopper et le compilateur (1952). Mathématicienne et officière de la marine américaine, Grace Hopper écrit le premier *compilateur*, programme qui traduit un texte lisible en langage machine. Elle défend l'idée qu'on doit pouvoir programmer avec des mots proches de l'anglais, ce qui conduit à COBOL. On lui attribue aussi l'origine du mot *bug* : en 1947, une mite est retrouvée coincée dans un relais du Harvard Mark II.

Les grandes familles de langages. Chaque langage apporte une idée, appelée un *paradigme* :

- *impératif* : une suite d'instructions qui modifient l'état de la mémoire (FORTRAN, C, Pascal, Python) ;
- *fonctionnel* : un programme est une composition de fonctions (LISP, Haskell, OCaml) ;
- *objet* : on regroupe données et fonctions dans des objets qui communiquent (Simula, Smalltalk, C++, Java).

Beaucoup de langages, dont Python, mélangent plusieurs paradigmes.

Année	Langage	Créateur	Apport / usage
1957	FORTRAN	John Backus (IBM)	Premier langage de haut niveau largement utilisé, pour le calcul scientifique, toujours employé en météo.
1958	LISP	John McCarthy	Programmation fonctionnelle ; langage de référence de l'IA jusque dans les années 1980.
1959	COBOL	Comité, dont Grace Hopper	Gestion ; des milliards de lignes tournent encore dans les banques.
1960	ALGOL 60	Comité international	Blocs, récursivité : ancêtre de C, Pascal, Java.
1964	BASIC	Kemeny et Kurtz	Conçu pour enseigner la programmation aux étudiants.
1967	Simula	Dahl et Nygaard	Premier langage à objets.
1970	Pascal	Niklaus Wirth	Conçu pour enseigner la programmation structurée.
1972	C	Dennis Ritchie	Écrire UNIX ; proche de la machine, toujours au cœur des systèmes d'exploitation.
1974	SQL	Chamberlin et Boyce	Interroger les bases de données (voir le chapitre dédié).
1983	C++	Bjarne Stroustrup	C avec objets ; jeux vidéo, logiciels performants.
1991	Python	Guido van Rossum	Lisibilité, simplicité ; science des données, IA, enseignement.
1995	Java	James Gosling (Sun)	« Écrire une fois, exécuter partout » ; entreprises, Android.
1995	JavaScript	Brendan Eich	Rend les pages Web interactives ; langage du navigateur.
2009	Go	Google (Pike, Thompson...)	Simplicité et concurrence pour les serveurs.
2010	Rust	Graydon Hoare (Mozilla)	Vitesse du C et sécurité de la mémoire garantie à la compilation.
2014	Swift	Apple	Applications iOS et macOS.

Ken Thompson et Dennis Ritchie : UNIX et C. Aux laboratoires Bell, ils créent le système d'exploitation *UNIX* (1969) puis le langage C (1972), dans lequel UNIX est réécrit. Le système devient alors portable d'une machine à l'autre. UNIX est l'ancêtre de Linux, de macOS et d'Android, et le C a inspiré C++, Java, JavaScript et Python.

Guido van Rossum : Python (1991). Ce programmeur néerlandais crée Python pendant les vacances de Noël 1989, en s'inspirant du langage ABC. Son nom vient des *Monty Python*, pas du serpent. Ses choix : une syntaxe lisible où l'indentation structure le code, et une grande bibliothèque de fonctions. Il est devenu l'un des langages les plus utilisés au monde, en particulier en science des données et en intelligence artificielle.

Rust et les langages récents. Les langages comme C et C++ laissent le programmeur gérer la mémoire, ce qui provoque des failles de sécurité fréquentes. Rust, apparu en 2010 et stable depuis 2015, garantit à la compilation l'absence de la plupart de ces erreurs,

sans ralentir le programme : il est adopté pour des systèmes critiques, y compris dans le noyau Linux depuis 2022. Il illustre la tendance actuelle : des langages qui aident le programmeur à écrire des programmes *sûrs*.

Le logiciel libre. En 1983, Richard Stallman lance le projet GNU pour que chacun puisse étudier, modifier et partager le code source des programmes. En 1991, l'étudiant finlandais Linus Torvalds publie le noyau *Linux* qui, associé aux outils GNU, forme un système d'exploitation libre : les langages et outils libres (Python, Linux, Rust) ont beaucoup contribué au développement de l'informatique moderne.

1.4 De nos jours... l'intelligence artificielle

Les débuts de l'intelligence artificielle. En 1950, Alan Turing publie un article qui commence par la question « Les machines peuvent-elles penser ? » et propose un test : une machine est dite intelligente si, lors d'une conversation écrite, un humain ne parvient pas à la distinguer d'une personne. L'expression « intelligence artificielle » est proposée en 1956 par John McCarthy lors d'un séminaire à Dartmouth. Les chercheurs se montrent très optimistes, puis déçus : les progrès sont bien plus lents que prévu.

Des succès spectaculaires. En 1997, l'ordinateur *Deep Blue* d'IBM bat le champion du monde d'échecs Garry Kasparov, en calculant des millions de positions par seconde. Mais le jeu de go, beaucoup plus complexe, résiste jusqu'en 2016, quand *AlphaGo* bat Lee Sedol. Cette fois, le programme n'a pas été programmé avec des règles de jeu : il a *appris* en jouant contre lui-même.

L'apprentissage automatique. Cette révolution repose sur les *réseaux de neurones*, dont les principes sont anciens, mais qui ne fonctionnent bien que depuis les années 2010, pour trois raisons : de très grandes quantités de données disponibles grâce à Internet, la puissance de calcul des processeurs graphiques (GPU), et les travaux de chercheurs comme Geoffrey Hinton, Yann LeCun et Yoshua Bengio, récompensés par le prix Turing en 2018. Au lieu d'écrire des règles, on présente des millions d'exemples à un programme, qui ajuste lui-même ses paramètres.

Les IA génératives. En 2017, des chercheurs de Google proposent l'architecture *Transformer*. Elle permet d'entraîner de très grands modèles de langage (*LLM*) sur d'énormes quantités de textes : ils apprennent à prédire le mot suivant, et par ce biais produisent des textes cohérents, du code, des images ou des sons. Avec la mise en ligne de ChatGPT fin 2022, ces outils touchent des centaines de millions de personnes. Ils soulèvent de nombreuses questions : fiabilité des réponses (ils peuvent affirmer des choses fausses avec aplomb), droits d'auteur, biais hérités des données, consommation d'énergie, effets sur l'école et sur les métiers, y compris celui de programmeur.

À retenir. L'informatique est une science récente, née de la rencontre des mathématiques, de la logique et de l'électronique. Les thèmes de cette année (représentation des nombres et des données, algorithmes, langages, architecture des machines, réseaux, intelligence artificielle) sont tous issus de cette histoire, et ont été imaginés par des personnes de cultures, d'époques et de parcours très différents.

2 Représentation d'un nombre en machine

2.1 Ecriture d'un entier dans une base quelconque $b \geq 2$

De nos jours tous les êtres humains, dès leur plus jeune âge, apprennent à compter sur leur dix doigts. Nous avons donc tous été familiarisés avec les nombres entiers, en base 10.

Les nombres entiers que nous manipulons se décomposent en unités, dizaines, centaines,

Mais on pourrait aussi bien utiliser la base 12 (utilisée en Egypte antique), la base 20 (utilisée par les Mayas et les Aztèques), la base 16 (système hexadécimal, utilisé en informatique) ou la base 60 (utilisé par les Sumériens au III^e millénaire av. J.-C. puis par les Babyloniens au II^e millénaire av. J.-C.). D'une manière générale, si b est un nombre entier positif, tout nombre entier x peut s'écrire :

$$(x)_{10} = \sum_{n=0}^N a_n \times b^n$$

où tous les coefficients a_n prennent leurs valeurs dans l'ensemble $\{0, 1, \dots, b-1\}$.

Exemples:

- $(10)_{10} = \mathbf{1} \times 10^1 + \mathbf{0} \times 10^0$
c'est-à-dire que: $b = 10$, $N = 1$, $a_0 = 0$ et $a_1 = 1$
- $(27)_{10} = \mathbf{2} \times 10^1 + \mathbf{7} \times 10^0$
c'est-à-dire que: $b = 10$, $N = 1$, $a_0 = 7$ et $a_1 = 2$
- $(3047)_{10} = \mathbf{3} \times 10^3 + \mathbf{0} \times 10^2 + \mathbf{4} \times 10^1 + \mathbf{7} \times 10^0$
c'est-à-dire que: $b = 10$, $N = 3$, $a_0 = 7$, $a_1 = 4$, $a_2 = 0$ et $a_3 = 3$

2.2 Ecriture d'un entier d'une base quelconque vers une autre base

- On part d'un entier d écrit en base 10 et on souhaite l'écrire en base quelconque, $b \in \mathbb{N}$.

Exemples:

- On veut écrire le nombre 17 pris en base 10, dans la base 8 (base octal):
 $(17)_{10} = \mathbf{2} \times 8^1 + \mathbf{1} \times 8^0 = (21)_8$
- On veut écrire le nombre 41 pris en base 10, dans la base 7 :
 $(41)_{10} = \mathbf{5} \times 7^1 + \mathbf{6} \times 7^0 = (56)_7$

La méthode consiste à effectuer des divisions euclidiennes successives, par b :

$$\begin{aligned}d &= b \times q_0 + r_0, \\q_0 &= b \times q_1 + r_1, \\&\dots = \dots \\q_{n-1} &= b \times 0 + r_n.\end{aligned}$$

On a alors : $(d)_{10} = (r_n \dots r_1 r_0)_b$

- Si l'entier est écrit dans une autre base que 10, alors on peut d'abord l'écrire en base 10, puis le transcrire dans la base souhaitée.

Exemples:

- On veut écrire le nombre 17 pris en base 3, dans la base 8 (base octal):
 $(122)_3 = 1 \times 3^2 + 2 \times 3^1 + 2 \times 3^0 = (17)_{10} = 2 \times 8^1 + 1 \times 8^0 = (21)_8$
- On veut écrire le nombre 41 pris en base 5, dans la base 6 :
 $(41)_5 = 4 \times 5^1 + 1 \times 5^0 = (21)_{10} = 3 \times 6^1 + 3 \times 6^0 = (33)_6$

2.3 Règles d'addition des nombres binaires

Les règles pour effectuer des additions dans l'ensemble des nombres binaires:

$$\begin{array}{rcl}0 & + & 0 = 0 \\1 & + & 0 = 1 \\0 & + & 1 = 1 \\1 & + & 1 = 10\end{array}$$

Exemples:

$(101)_2 + (11)_2$

retenues	1	1	1	
	0	1	0	1
+	0	0	1	1
	1	0	0	0

soit $5 + 3 = 8$

0 + 0 + 1	1 + 0 + 1	0 + 1 + 1	1 + 1
= 1	= 10	= 10	= 10

on calcule colonne par colonne, de droite à gauche

$$(1001)_2 + (110)_2$$

$$\begin{array}{rcccc}
 & 1 & 0 & 0 & 1 \\
 + & 0 & 1 & 1 & 0 \\
 \hline
 & 1 & 1 & 1 & 1
 \end{array}
 \quad \text{soit } 9 + 6 = 15$$

$1 + 0 = 1$ $0 + 1 = 1$ $0 + 1 = 1$ $1 + 0 = 1$

aucune retenue : on calcule colonne par colonne, de droite à gauche

$$(1100)_2 - (111)_2$$

$$\begin{array}{rcccc}
 & & +10 & +10 & +10 \\
 & 1 & 1 & 0 & 0 \\
 - & 0 & 1 & 1 & 1 \\
 & +1 & +1 & +1 & \\
 \hline
 & 0 & 1 & 0 & 1
 \end{array}
 \quad \text{soit } 12 - 7 = 5$$

$1 - 1 = 0$ $11 - 10 = 1$ $10 - 10 = 0$ $10 - 1 = 1$

Quand le chiffre du haut est trop petit, on lui ajoute 10 (=2) et on ajoute 1 au chiffre du bas de la colonne suivante.

2.4 Représentation en machine d'un entier positif

Le fonctionnement des ordinateurs est assurée par des courants électriques qui "passent" (valeur 1) ou "ne passent pas" (valeur 0) dans des circuits internes à l'ordinateur.

Les mathématiciens, qui étaient au fait des systèmes de numération, ont envisagé la représentation en base 2 des nombres. Tout nombre entier positif x en base 10 peut s'écrire aussi en base 2, on a alors:

$$(x)_{10} = \sum_{n=0}^N a_n \times 2^n = (a_n a_{n-1} \dots a_1 a_0)_2$$

Exemples:

- $(5)_{10} = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$
c'est-à-dire que: $b = 2$, $N = 2$, $a_0 = 1$, $a_1 = 0$ et $a_2 = 1$. On a donc: $(5)_{10} = (101)_2$.
- $(19)_{10} = 1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$
c'est-à-dire que: $b = 2$, $N = 4$, $a_0 = 1$, $a_1 = 1$, $a_2 = 0$, $a_3 = 0$ et $a_4 = 1$. On a donc:

$$(19)_{10} = (10011)_2.$$

- $(47)_{10} = 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$
c'est-à-dire que: $b = 2$, $N = 5$, $a_0 = 1$, $a_1 = 1$, $a_2 = 1$, $a_3 = 1$, $a_4 = 0$ et $a_5 = 1$. On a donc: $(47)_{10} = (101111)_2$.

2.5 Représentation en machine d'un entier relatif

Dès le début des ordinateurs, s'est posée la question de la représentation des nombres entiers *négatifs*. Tout d'abord il faut garder en tête que l'ordinateur lit un entier sur *un nombre de bits (contraction de Binay Digit) fixé*. On dit que l'entier est codé sur ce nombre de bits.

Par exemple sur 4 bits:

- le premier bit de gauche représentera le signe (0 pour un positif et 1 pour un négatif).
- les bits suivants, c'est-à-dire 3 bits servent à coder l'entier. Le plus grand sera $(0111)_2$ c'est-à-dire $(7)_{10}$ et le plus petit sera le codage de $(-7)_{10}$, en passant par $(0000)_2$

Le mauvais choix de codage des entiers négatifs:

Le premier bit est mis à 1 et les 3 bits suivants sont utilisés pour coder la distance à zéro du nombre. Par exemple on coderait -5 de la manière suivante 1101.

Essayons alors de faire l'opération $(-1)_{10} + (-5)_{10}$ en utilisant les écritures binaires de chacun des termes. On devra donc additionner $(1001)_2$ à $(1101)_2$, ce qui ferait $(10110)_2$. Comme les entiers sont codés ici sur 4 bits (on ne lit pas le premier bit de gauche) et on aurait donc comme résultat: $(0110)_2$ ce qui est un nombre positif...Oups !

Mais il y avait en amont un problème...On a deux codages possibles de 0: $(0000)_2$ et $(1000)_2$?

Le bon choix de codage des entiers négatifs:

Illustrons la démarche sur un exemple. On part du nombre entier négatif -1 . On va d'abord coder sur 4 bits le nombre entier positif 1, on a:

$$(1)_{10} = (0001)_2$$

Puis on inverse le nombre binaire obtenu, bit par bit:

$$(1110)_2$$

Puis on ajoute 1:

$$(\mathbf{1111})_2$$

On effectue la même opération avec le nombre -5 :
 On écrit l'entier positif 5 en binaire :

$$(0101)_2$$

On inverse bit par bit:

$$(1010)_2$$

On ajoute 1:

$$(\mathbf{1011})_2$$

Pour effectuer le calcul $(-1)_{10} + (-5)_{10}$, on va utiliser les codages binaires précédents (en gras) des nombres -1 et -5 :

$$(-1)_{10} + (-5)_{10} = (1111)_2 + (1011)_2 = (1010)_2$$

Le résultat est $(1010)_2$. Si on souhaite vérifier que c'est le bon résultat alors on va appliquer les opérations d'inversion et d'ajout de 1 au nombre $(1010)_2$, on a:

$$(1010)_2 \xrightarrow{\text{inversion bit à bit}} (0101)_2 \xrightarrow{+1} (0110)_2$$

Et on remarque que $(0110)_2$ est le codage de 6 donc cela veut dire que $(1010)_2$ est bien le codage de -6 .

Remarques:

-> Il faut s'assurer que le nombre de bits est suffisant pour le codage. Si on souhaite faire l'opération $(-4)_{10} + (-5)_{10}$ alors il faudra coder sur au moins 5 bits car la somme sera -9 . Sur 5 bits le plus grand nombre positif qu'on pourra coder sera 01111 c'est-à-dire $(15)_{10}$...Ce qui suffira.

-> La méthode qui consiste à inverser bit à bit et à ajouter $+1$ est appelée **la méthode du complément à deux**.

-> L'arithmétique modulaire

Si on reprend l'exemple du codage de -5 sur **4 bits** pour lequel on a trouvé **1011**. On constate que **1011** est aussi l'écriture en binaire du nombre $(11)_{10}$.

Et on a:

$$5 + 11 = 16$$

C'est-à-dire:

$$5 + 11 = 2^4$$

Ou encore:

$$5 + 11 \equiv 0[2^4]$$

Ce qui se dit:

- 5 plus 11 est égal à 0 modulo 2^4 .
- 11 est l'opposé de 5 dans l'arithmétique modulo 2^4 ; exactement comme -5 est l'opposé de 5 dans l'arithmétique standard:

$$5 + (-5) = 0$$

Un autre exemple:

On me demande le codage de -13 sur **5 bits** (4 ne suffiraient pas...). Le bit de gauche est pour le signe, il reste donc 4 bits avec lesquels je peux coder les entiers

$$-2^4 + 1, \dots, 0, \dots, 2^4 - 1$$

c'est-à-dire:

$$-15, \dots, 0, \dots, 15$$

Je pars de 15, je me pose la question suivante:

$$13 + ? = 2^5$$

On trouve:

$$13 + 19 = 2^5$$

Et l'écriture binaire de $(19)_{10}$ est $(10011)_2$ qui correspond aussi au codage de -13 sur 5 bits et qu'on aurait trouvé en utilisant le complément à deux de 13...

Encore un autre exemple:

On me demande le codage de -37 . Comme $(37)_{10} = (100101)_2$ s'écrit déjà sur 6 bits, il faut un bit de plus pour le signe, soit **7 bits** (6 ne suffiraient pas: on ne pourrait coder que de -31 à 31). Avec 7 bits, il reste 6 bits pour les entiers

$$-2^6 + 1, \dots, 0, \dots, 2^6 - 1$$

c'est-à-dire:

$$-63, \dots, 0, \dots, 63$$

Je me pose la question suivante:

$$37 + ? = 2^7$$

On trouve:

$$37 + 91 = 128 = 2^7$$

Et l'écriture binaire de $(91)_{10}$ est $(1011011)_2$ qui est le codage de -37 sur 7 bits.

Vérifions avec le complément à deux de 37: on écrit 37 sur 7 bits $(0100101)_2$, on inverse bit par bit $(1011010)_2$, puis on ajoute 1: $(1011011)_2$. On retrouve bien le même résultat.

→ N.B.: La bonne terminologie est **méthode du complément à 2^n** où n est le nombre de bits sur lequel on code. On utilise comme codage de l'opposé d'un entier x positif, le codage du nombre $2^n - x$; c'est le complément de x dans l'arithmétique modulo 2^n . On a:

$$x + (2^n - x) = 2^n \equiv 0[2^n]$$

La méthode qui consiste à inverser bit à bit et à ajouter 1 revient à faire $((2^n - 1) - x) + 1$. Et on a bien

$$((2^n - 1) - x) + 1 = 2^n - x$$

qui est le complément à deux de x .

2.6 Représentation en machine d'un réel

Nous avons vu comment sont représentés les entiers relatifs dans un ordinateur. Nous avons notamment vu que tout entier relatif n peut s'écrire en base 2 sous la forme:

$$(n)_{10} = \sum_{k=0}^K a_k \times 2^k$$

On remarque que la somme (désignée par $\sum$) commence par 0 et se termine par $K > 0$.

Nous allons maintenant voir comment sont représentés les nombres décimaux (à virgules) en base 2. Tout nombre décimal x peut se décomposer en puissance de 10, et contrairement aux nombres entiers, il peut y avoir des puissances négatives de 10.

$$(n)_{10} = \sum_{k=-K_1}^{K_2} a_k \times 10^k$$

où $K_1 > 0$ et $K_2 > 0$.

Maintenant il s'agit de voir comment on décompose, en base 2, les nombres décimaux.

Exemples:

1. Nous souhaitons représenter le nombre 5.1875 en binaire. La partie entière 5 s'écrit 101 en binaire. Qu'en est-il de la partie décimale ?

Dans ce qui suit nous détaillons une méthode qui permet d'identifier les coefficients 0 ou 1 dans le développement binaire de 0.1875.

$$\begin{aligned} 0.1875 \times 2 &= 0 + 0.375 \\ 0.375 \times 2 &= 0 + 0.75 \\ 0.75 \times 2 &= 1 + 0.5 \\ 0.5 \times 2 &= 1 + 0 \end{aligned}$$

Ce qui permet d'écrire:

$$(5.1875)_{10} = (101.0011)_2$$

2. On essaie à nouveau, avec le nombre $(4.125)_{10}$. On a:

$$\begin{aligned} 0.125 \times 2 &= 0 + 0.250 \\ 0.250 \times 2 &= 0 + 0.5 \\ 0.5 \times 2 &= 1 + 0 \end{aligned}$$

Ce qui permet d'écrire:

$$(4.125)_{10} = (100.001)_2$$

3. Un dernier exemple où les choses ne se passent pas comme on le souhaiterait... :

On essaie à nouveau, avec le nombre $(0.1)_{10}$. On a:

$$\begin{aligned}
 0.1 \times 2 &= 0 + 0.2 \\
 0.2 \times 2 &= 0 + 0.4 \\
 0.4 \times 2 &= 0 + 0.8 \\
 0.8 \times 2 &= 1 + 0.6 \\
 0.6 \times 2 &= 1 + 0.2 \\
 0.2 \times 2 &= 0 + 0.4 \\
 0.4 \times 2 &= 0 + 0.8 \\
 0.8 \times 2 &= 1 + 0.6 \\
 0.6 \times 2 &= 1 + 0.2 \\
 0.2 \times 2 &= 0 + 0.4 \\
 0.4 \times 2 &= 0 + 0.8 \\
 &\dots = \dots
 \end{aligned}$$

On voit que la décomposition de 0.1 en base 2 ne se termine pas. Donc on aura uniquement une approximation de 0.1 en machine.

Remarque: bien sûr il est possible de retrouver une représentation décimale en base 10 à partir d'une représentation en binaire.

Partons de $(100.0101)_2$. On peut alors écrire:

$$(100.0101)_2 = 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4}$$

c'est-à-dire:

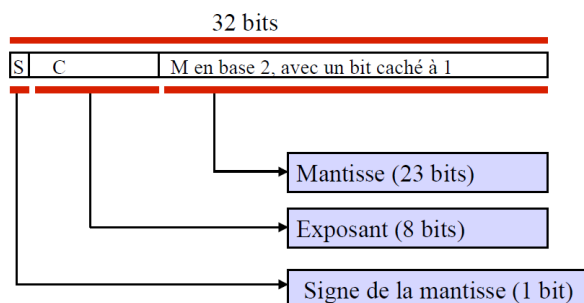
$$(100.0101)_2 = 4 + \frac{1}{2^2} + \frac{1}{2^4}$$

donc:

$$(100.0101)_2 = 4 + \frac{1}{4} + \frac{1}{16} = (4.3125)_{10}$$

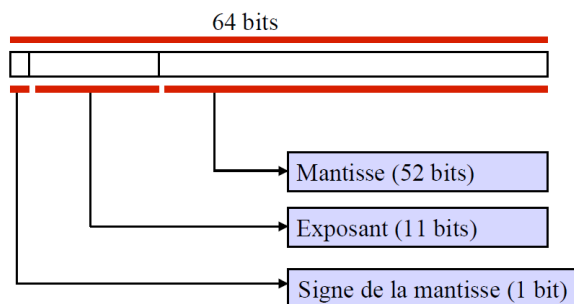
Le standard IEEE 754

Au début des années 1980, l'association IEEE (Institute of Electrical and Electronics Engineers) a édité un standard (qui porte le numéro 754) et qui concerne la représentation des réels dans les ordinateurs. Selon la norme IEEE 754, les réels sont représentés sur 32 bits de la manière suivante:



On parle de nombres flottants en *simple précision*.

On peut aussi représenter les nombres réels sur 64 bits:



On parle de nombres flottants en *double précision*.

Remarques:

- Le bit de signe prend la valeur 0 pour les nombres positifs ou 1 pour les nombres négatifs.
- L'exposant sera représenté sur 8 bits et donc il y a $2^8 = 256$ exposants possibles (de 0 qui correspond à 00000000 à 255 qui correspond à 11111111. Dans le cas du 64 bits, il y aura $2^{11} = 2048$ exposants possibles (de 0 qui correspond à 000000000000 à 2047 qui correspond à 111111111111).
- Les exposants 00000000 et 11111111 (ou 000000000000 et 111111111111 en 64 bits) sont réservés pour le zéro et l'infini (et le NaN c'est-à-dire Not A Number).
- Il a été décidé de ne représenter que les nombres utilisant des puissances de 2 allant de 2^{-126} à 2^{+127} , pas moins et pas plus (allant de 2^{-1022} à 2^{+1023} en 64 bits).
- Les **puissances négatives** de 2 sont représentées de 1 à 126 (de 1 à 1023 en 64 bits), et les **puissances positives** de 2 sont représentées de 127 à 254 (de 1024 à 2046 en 64 bits). Pour ce faire on ajoute à la puissance de 2 (positive ou négative) la quantité 127 (ce sera 1023 en 64 bits). Par exemple, en 32 bits, le nombre 1.011×2^{17} sera codé comme étant $1.011 \times 2^{17+127}$ ou encore le nombre 1.10111×2^{-5} sera codé comme étant le nombre $1.10111 \times 2^{-5+127}$.
- La mantisse est codé sur 23 bits (sur 53 bits pour une représentation en 64 bits) et correspond à la partie qui vient après le 1.

Exemples :

1. On souhaite représenter le réel $(10.78125)_{10}$ en 32 bits.

- *Etape 1* On écrit le nombre réel en binaire. Après calcul, on obtient:

$$(10.78125)_{10} = (1010, 11001)_2$$

- *Etape 2* On écrit ce nombre binaire sous la forme $1... \times 2^...$. On obtient ici:

$$1.01011001 \times 2^3$$

- *Etape 3* On décale l'exposant de +127 vers la droite et donc le nombre qui va être codé est

$$1.01011001 \times 2^{3+127}$$

- Le nombre est positif donc on stockera le chiffre 0 comme valeur du bit de signe.
- Le codage de l'exposant 130, sur 8 bits est : 10000010.
- La partie décimale du coefficient devant la puissance de 2 est 010110010...0

- Finalement le codage en nombre flottant sur 32 bits de $(10.78125)_{10}$ est:

$$01000001001011001000000000000000$$

2. Si on devait représenter le réel négatif $(-10.78125)_{10}$ en 32 bits, on travaillerait sur le nombre réel positif $(10.78125)_{10}$ comme précédemment et on concluerait que le codage en nombre flottant sur 32 bits de $(-10.78125)_{10}$ est:

$$11000001001011001000000000000000$$

3. On considère le nombre flottant suivant en simple précision:

$$11000001011110110000000000000000$$

On va déterminer le nombre en base 10 dont il est le codage.

- On sait qu'il s'agit d'un nombre négatif car le bit de signe vaut 1.
- L'exposant est 130 ce qui veut dire que la puissance de 2 cherché est $130 - 127 = 3$.
- Le nombre codé (abstraction faite du signe) est donc l'opposé du nombre

$$1.1111011 \times 2^3$$

c'est-à-dire l'opposé du nombre

$$\left(\frac{1}{2^0} + \frac{1}{2^1} + \frac{1}{2^2} + \frac{1}{2^3} + \frac{1}{2^4} + \frac{0}{2^5} + \frac{1}{2^6} + \frac{1}{2^7}\right) \times 2^3$$

ou encore

$$(1 + 0.5 + 0.25 + 0.125 + 0.0625 + 0 + 0.015625 + 0.0078125) \times 2^3$$

Pour conclure, le codage

$$11000001011110110000000000000000$$

est celui du nombre

$$-15.6875$$